



Tarea 1

Integrantes: Renato Spröhnle y Victor Ruiz

Pregunta 1: Planificación de producción de aleaciones

Conjuntos

$\mathcal{J} = \{1, \dots, J\}$: conjunto de tipos de aleaciones de oro.

$\mathcal{M} = \{1, \dots, M\}$: conjunto de metales base disponibles.

$\mathcal{T} = \{1, \dots, T\}$: conjunto de períodos del horizonte de planificación.

Parámetros

d_{jt} : demanda máxima (gramos) de la aleación $j \in \mathcal{J}$ en el período $t \in \mathcal{T}$.

p_{jt} : precio de venta (\$/gramo) de la aleación j en el período t .

r_m : índice de calidad del metal $m \in \mathcal{M}$.

q_j : calidad promedio ponderada mínima exigida a la aleación j .

a_{mt} : gramos del metal m que se reciben al inicio del período t .

$\hat{\gamma}_m$: inventario inicial (gramos) del metal m (al inicio del período 1).

γ_j : inventario inicial (gramos) de la aleación j .

$\hat{h}_{mt}$: costo (\$/gramo) de mantener el metal m en inventario desde t hasta $t + 1$.

h_{jt} : costo (\$/gramo) de mantener la aleación j en inventario desde t hasta $t + 1$.

v_m : valor (\$/gramo) del metal m que quede en inventario al final del horizonte.

Variables de decisión

Todas las variables son continuas y no negativas (gramos o dólares, según corresponda).

x_{mjt} : gramos del metal m utilizados para producir la aleación j en el período t .

y_{jt} : gramos producidos de la aleación j en el período t .

w_{jt} : gramos vendidos de la aleación j en el período t .

I_{mt}^M : gramos del metal m en inventario al *final* del período t .

I_{jt}^A : gramos de la aleación j en inventario al *final* del período t .

Función objetivo

Maximizamos la utilidad total: ingresos por ventas menos costos de mantener inventario (de metal y aleación), más el valor de salvataje del metal al final del horizonte. (Los parámetros a_{mt} y los inventarios iniciales son datos y no se incluyen como costos, pues ya ocurrieron o son ingresos del sistema.)

$$\begin{aligned} \text{máx} \quad & \underbrace{\sum_{j \in \mathcal{J}} \sum_{t \in \mathcal{T}} p_{jt} w_{jt}}_{\text{ingresos por ventas}} - \underbrace{\sum_{m \in \mathcal{M}} \sum_{t=1}^{T-1} \hat{h}_{mt} I_{mt}^M}_{\text{costo inv. de metales}} - \underbrace{\sum_{j \in \mathcal{J}} \sum_{t=1}^{T-1} h_{jt} I_{jt}^A}_{\text{costo inv. de aleaciones}} + \underbrace{\sum_{m \in \mathcal{M}} v_m I_{mT}^M}_{\text{valor de salvataje}}. \end{aligned}$$

Restricciones

(R1) Balance de inventario de metales. Para cada metal $m \in \mathcal{M}$ y período $t \in \mathcal{T}$:

$$I_{mt}^M = I_{m,t-1}^M + a_{mt} - \sum_{j \in \mathcal{J}} x_{mjt}, \quad \text{con } I_{m,0}^M := \hat{\gamma}_m.$$

Es decir, lo que queda en bodega al final de t es el inventario previo más lo recibido, menos lo consumido en producción.

(R2) Balance de inventario de aleaciones. Para cada $j \in \mathcal{J}$, $t \in \mathcal{T}$:

$$I_{jt}^A = I_{j,t-1}^A + y_{jt} - w_{jt}, \quad \text{con } I_{j,0}^A := \gamma_j.$$

(R3) Composición de cada aleación. La suma de los gramos de metales usados para producir la aleación j debe ser *exactamente* y_{jt} :

$$\sum_{m \in \mathcal{M}} x_{mjt} = y_{jt}, \quad \forall j \in \mathcal{J}, t \in \mathcal{T}.$$

(R4) Calidad promedio ponderada. La calidad media ponderada de la aleación j producida en t debe ser al menos q_j . Cuando $y_{jt} > 0$ equivale a $\sum_m r_m x_{mjt}/y_{jt} \geq q_j$; al multiplicar por y_{jt} y usar la restricción (R3), la linealización correcta es:

$$\sum_{m \in \mathcal{M}} (r_m - q_j) x_{mjt} \geq 0, \quad \forall j \in \mathcal{J}, t \in \mathcal{T}.$$

(Si $y_{jt} = 0$ la desigualdad se cumple trivialmente porque todos los $x_{mjt} = 0$ por (R3).)

(R5) Cota de demanda. No se puede vender más que la demanda máxima:

$$w_{jt} \leq d_{jt}, \quad \forall j \in \mathcal{J}, t \in \mathcal{T}.$$

(R6) No negatividad.

$$x_{mjt}, y_{jt}, w_{jt}, I_{mt}^M, I_{jt}^A \geq 0, \quad \forall m, j, t.$$

Observación. Las variables I_{mt}^M e I_{jt}^A no son estrictamente necesarias (podrían despejarse recursivamente de los balances); se mantienen para mayor claridad y porque aparecen tanto en la FO como en la restricción de salvataje ($v_m I_{mT}^M$). El modelo completo es lineal continuo, tal como pide el enunciado.

Pregunta 2: Planificación de sesiones de estudio

Conjuntos

$\mathcal{T} = \{1, \dots, T\}$: bloques horarios del día, ordenados.

$\mathcal{B} \subseteq \mathcal{T}$: bloques en los que *no* se puede estudiar.

Parámetros

h_t : costo fijo de *iniciar* una sesión al comienzo del bloque t .

c_t : costo variable por bloque de estar estudiando en el bloque t .

H : número mínimo total de bloques de estudio en el día.

U : duración mínima de cualquier sesión (bloques).

D : descanso mínimo entre el término de una sesión y el inicio de la siguiente (bloques).

K : duración mínima requerida para *al menos una* sesión del día, con $K > U$.

Variables de decisión

Todas son binarias.

x_t : 1 si el estudiante está estudiando en el bloque t , 0 en caso contrario.

s_t : 1 si *inicia* una sesión al comienzo del bloque t .

e_t : 1 si *termina* una sesión durante el bloque t (es decir, t es el último bloque de una sesión).

z_t : 1 si la sesión que inicia en el bloque t dura al menos K bloques (variable auxiliar para la sesión “larga”).

Función objetivo

Se minimiza el costo total: costos fijos por iniciar sesiones + costos variables por bloque estudiado.

$$\text{mín} \sum_{t \in \mathcal{T}} h_t s_t + \sum_{t \in \mathcal{T}} c_t x_t.$$

Restricciones

(R1) Relación entre x , s y e . Notemos que $s_t = 1$ equivale a “estoy estudiando en t pero no en $t - 1$ ” y $e_t = 1$ equivale a “estoy estudiando en t pero no en $t + 1$ ”. Con la convención $x_0 = x_{T+1} = 0$, podemos caracterizar estas variables mediante las siguientes dos familias de desigualdades:

$$s_t \geq x_t - x_{t-1}, \quad \forall t \in \mathcal{T}, \quad x_0 := 0, \quad (1)$$

$$s_t \leq x_t, \quad \forall t \in \mathcal{T}, \quad (2)$$

$$s_t \leq 1 - x_{t-1}, \quad \forall t \in \mathcal{T}, \quad x_0 := 0, \quad (3)$$

$$e_t \geq x_t - x_{t+1}, \quad \forall t \in \mathcal{T}, \quad x_{T+1} := 0, \quad (4)$$

$$e_t \leq x_t, \quad \forall t \in \mathcal{T}, \quad (5)$$

$$e_t \leq 1 - x_{t+1}, \quad \forall t \in \mathcal{T}, \quad x_{T+1} := 0. \quad (6)$$

En conjunto, esas desigualdades fuerzan $s_t = \max\{0, x_t - x_{t-1}\}$ y $e_t = \max\{0, x_t - x_{t+1}\}$.¹

(R2) Bloques prohibidos.

$$x_t = 0, \quad \forall t \in \mathcal{B}.$$

(R3) Total mínimo de bloques de estudio.

$$\sum_{t \in \mathcal{T}} x_t \geq H.$$

(R4) Duración mínima U de cada sesión. Si se inicia una sesión en t , entonces se estudia durante los U bloques consecutivos $t, t+1, \dots, t+U-1$:

$$x_{t+k} \geq s_t, \quad \forall t \in \mathcal{T}, k = 0, 1, \dots, U-1 \text{ tal que } t+k \leq T.$$

Cuidado con los bordes: si $t > T - U + 1$, entonces t no alcanza a “caber” una sesión de largo U ; en ese caso prohibimos iniciar allí:

$$s_t = 0, \quad \forall t > T - U + 1.$$

(R5) Descanso mínimo D entre sesiones. Si termina una sesión en t ($e_t = 1$), no puede estudiarse durante los D bloques siguientes:

$$x_{t+k} \leq 1 - e_t, \quad \forall t \in \mathcal{T}, k = 1, 2, \dots, D \text{ tal que } t+k \leq T.$$

(R6) Al menos una sesión de duración $\geq K$. Introducimos z_t que “marca” si la sesión que inicia en t dura $\geq K$. Dos familias de restricciones:

$$z_t \leq s_t, \quad \forall t \in \mathcal{T}, \tag{7}$$

$$x_{t+k} \geq z_t, \quad \forall t \in \mathcal{T}, k = 0, 1, \dots, K-1 \text{ tal que } t+k \leq T. \tag{8}$$

Luego, imponemos que al menos una sesión sea larga:

$$\sum_{t=1}^{T-K+1} z_t \geq 1.$$

(Notar que z_t solo tiene sentido para $t \leq T - K + 1$; para los demás t podríamos fijarlos a cero, o la suma con esos índices simplemente no los considera.)

(R7) Dominio de las variables.

$$x_t, s_t, e_t, z_t \in \{0, 1\}, \quad \forall t \in \mathcal{T}.$$

Observación sobre los bordes. Las convenciones $x_0 := 0$ y $x_{T+1} := 0$ son claves. La primera hace que estudiar desde el bloque 1 se considere un inicio. La segunda implementa literalmente lo que dice el enunciado: “si se está estudiando durante el último bloque del día se considera que la sesión acaba durante ese último bloque”.

¹Para $t = 1$ se toma $x_0 = 0$, luego $s_1 = x_1$: si se empieza estudiando el día, se considera un inicio. Para $t = T$ se toma $x_{T+1} = 0$, luego $e_T = x_T$: si se está estudiando en el último bloque, la sesión termina en T , tal como indica el enunciado.

Pregunta 3

a. Solución gráfica

Consideremos el problema:

$$\text{máx } 2x_1 + x_2 \quad \text{s.a.} \quad x_1 + x_2 \leq 3,5, \quad 2x_1 + x_2 \leq 6, \quad x_1 \leq 2,5, \quad x_1, x_2 \geq 0.$$

Región factible (con las tres restricciones). Los vértices son $(0,0)$, $(2,5, 0)$, $(2,5, 1)$ y $(0, 3,5)$. Evaluando la función objetivo $z = 2x_1 + x_2$ en cada uno: $z(0,0) = 0$, $z(2,5, 0) = 5$, $z(2,5, 1) = 6$, $z(0, 3,5) = 3,5$. El óptimo es *único* y se alcanza en el vértice $(x_1^*, x_2^*) = (2,5, 1)$ con valor óptimo $z^* = 6$.

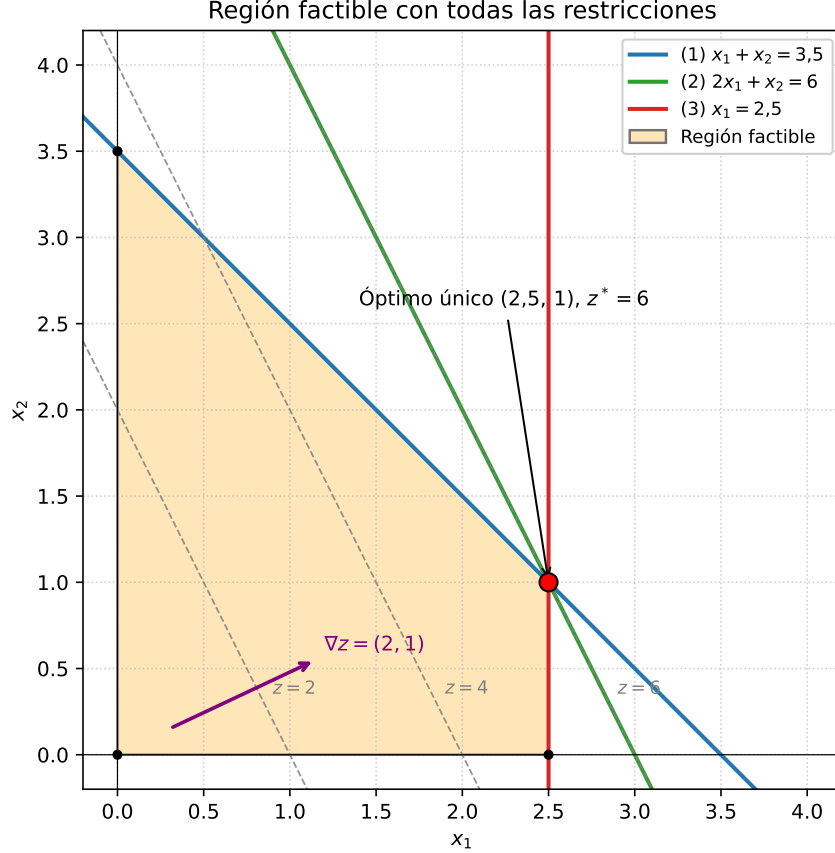


Figura 1: Región factible con las tres restricciones. El vector $\nabla z = (2,1)$ indica la dirección de crecimiento de la FO; la curva de nivel $z = 6$ toca la región exactamente en $(2,5, 1)$.

Eliminando la restricción $x_1 \leq 2,5$. Sin esta cota, los vértices pasan a ser $(0,0)$, $(3,0)$, $(2,5, 1)$ y $(0, 3,5)$. Los valores de z son ahora 0, 6, 6 y 3,5 respectivamente. Como el gradiente de la FO $(2,1)$ es paralelo al gradiente de la restricción $(2) 2x_1 + x_2 \leq 6$, la curva de nivel $z = 6$ coincide con la arista entre $(2,5, 1)$ y $(3,0)$. Por lo tanto aparecen *óptimos múltiples*: *cualquier* punto del segmento

$$\{(x_1, x_2) \in \mathbb{R}_+^2 : 2x_1 + x_2 = 6, 2,5 \leq x_1 \leq 3\}$$

es óptimo y el valor óptimo sigue siendo $z^* = 6$. Equivalentemente, el conjunto de óptimos es

$$\{(\lambda \cdot 2,5 + (1 - \lambda) \cdot 3, \lambda \cdot 1 + (1 - \lambda) \cdot 0) : \lambda \in [0, 1]\} = \{(3 - 0,5\lambda, \lambda) : \lambda \in [0, 1]\}.$$

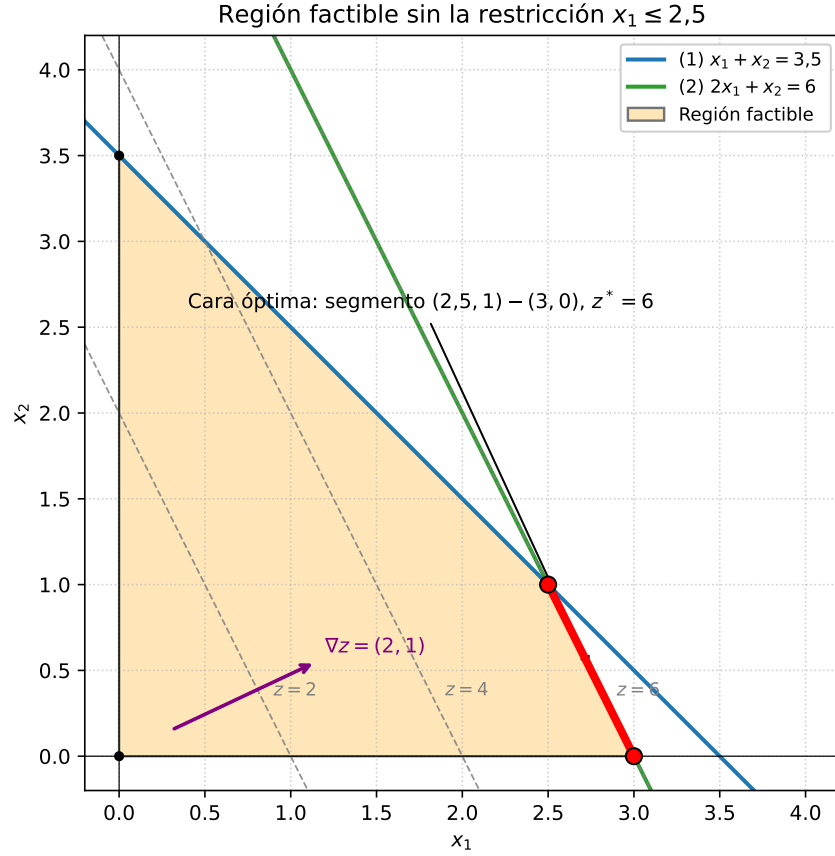


Figura 2: Región factible sin la restricción $x_1 \leq 2,5$. La cara óptima es el segmento que une $(2,5, 1)$ con $(3, 0)$.

b. Implementación computacional

1. Interpretación del modelo

Variables. $y_i \in \{0, 1\}$ indica si la bodega $i \in O$ es arrendada ($y_i = 1$) o no ($y_i = 0$). La variable continua $x_{ij} \geq 0$ representa las unidades de producto enviadas desde la bodega i al cliente $j \in D$.

Función objetivo. Se minimiza el costo total, compuesto por (i) el costo variable de transporte $\sum_{i,j} c_{ij}x_{ij}$ y (ii) el costo fijo de arriendo de las bodegas utilizadas $\sum_i F_i y_i$.

Restricciones.

- La restricción (4) $\sum_j x_{ij} \leq s_i y_i$ cumple dos funciones simultáneamente: (a) impone la capacidad máxima s_i cuando la bodega está arrendada ($y_i = 1$) y (b) prohíbe enviar desde una bodega no arrendada ($y_i = 0 \Rightarrow \sum_j x_{ij} \leq 0 \Rightarrow x_{ij} = 0 \forall j$). Esta es la “linealización” clásica del acoplamiento abrir/usar una instalación.
- La restricción (5) $\sum_i x_{ij} = d_j$ obliga a satisfacer *exactamente* la demanda de cada cliente.

2. Implementación en Python con Gurobi

El archivo `main.py` (adjunto) lee los archivos `oferta.csv` y `costos_demanda.csv` desde la carpeta de ejecución y resuelve el modelo anterior. El código no asume la cantidad de bodegas ni de clientes, detectando

ambos desde los encabezados y filas de los archivos. Detecta infactibilidad usando el estado GRB.INFEASIBLE (y GRB.INF_OR_UNBD). A continuación se adjunta el código.

```

1 """
2 ICS1113 - Optimizacion
3 Tarea 1, Pregunta 3b
4
5 Lee los archivos 'oferta.csv' y 'costos_demanda.csv' desde la carpeta
6 actual, construye el modelo de localizacion / transporte en Gurobi y
7 reporta la solucion optima (o informa infactibilidad).
8 """
9
10 import csv
11 import sys
12 from typing import Tuple, List, Dict, Any
13
14 import gurobipy as gp
15 from gurobipy import GRB
16
17
18 def leer_oferta(ruta: str) -> Tuple[List[str],
19                                     Dict[str, float], Dict[str, float]]:
20     """Devuelve (origenes, s, F) donde:
21     - origenes es la lista de IDs (como str) en el orden de aparicion,
22     - s[i] es la oferta maxima de la bodega i,
23     - F[i] es el costo fijo de arriendo de la bodega i.
24     """
25     origenes: List[str] = []
26     s: Dict[str, float] = {}
27     F: Dict[str, float] = {}
28     with open(ruta, mode="r", encoding="utf-8", newline="") as f:
29         reader = csv.reader(f)
30         next(reader, None) # saltar encabezado
31         for fila in reader:
32             if not fila or all(c.strip() == "" for c in fila):
33                 continue
34             i: str = fila[0].strip()
35             origenes.append(i)
36             s[i] = float(fila[1])
37             F[i] = float(fila[2])
38     return origenes, s, F
39
40
41 def leer_costos_demanda(ruta: str,
42                         origenes: List[str]) -> Tuple[List[str],
43                                                         Dict[Tuple[str, str], float],
44                                                         Dict[str, float]]:
45     """Devuelve (destinos, c, d) donde:
46     - destinos es la lista de IDs de clientes (como str),
47     - c[(i,j)] es el costo unitario de enviar de bodega i a cliente j,
48     - d[j] es la demanda del cliente j.
49     """
50     with open(ruta, mode="r", encoding="utf-8", newline="") as f:
51         reader = csv.reader(f)
52         filas: List[List[str]] = [fila for fila in reader if fila and any(
53             celda.strip() != "" for celda in fila)]
54
55     encabezado: List[str] = filas[0]
56     destinos: List[str] = [x.strip() for x in encabezado[1:]]
57
58     c: Dict[Tuple[str, str], float] = {}
59     d: Dict[str, float] = {}
60
61     for fila in filas[1:-1]:
62         i: str = fila[0].strip()

```

```

63         for k, j in enumerate(destinos):
64             c[(i, j)] = float(fila[k + 1])
65
66     fila_demanda: List[str] = filas[-1]
67     for k, j in enumerate(destinos):
68         d[j] = float(fila_demanda[k + 1])
69
70     return destinos, c, d
71
72
73 def main() -> None:
74     # ----- Lectura de datos -----
75     ruta_oferta: str = "oferta.csv"
76     ruta_costos: str = "costos_demanda.csv"
77
78     try:
79         origenes, s, F = leer_oferta(ruta_oferta)
80         destinos, c, d = leer_costos_demanda(ruta_costos, origenes)
81     except FileNotFoundError as e:
82         print(f"Error: {e}")
83         print("La tarea debe ser ejecutada desde la carpeta que contiene oferta.csv y
84             costos_demanda.csv")
85         sys.exit(1)
86
87     # ----- Modelo -----
88     modelo: gp.Model = gp.Model("Localizacion_Transporte")
89     modelo.setParam("OutputFlag", 0) # silenciar el log de Gurobi
90
91     # Variables
92     x = modelo.addVars(
93         origenes,
94         destinos,
95         lb=0.0,
96         vtype=GRB.CONTINUOUS,
97         name="x")
98     y = modelo.addVars(origenes, vtype=GRB.BINARY, name="y")
99
100     # Funcion objetivo: costos de transporte + costos fijos de arriendo
101     modelo.setObjective(
102         gp.quicksum(c[(i, j)] * x[i, j] for i in origenes for j in destinos)
103         + gp.quicksum(F[i] * y[i] for i in origenes),
104         GRB.MINIMIZE,
105     )
106
107     # (4) Capacidad + ligadura con la decision de arriendo
108     modelo.addConstrs(
109         (gp.quicksum(x[i, j] for j in destinos) <= s[i] * y[i] for i in origenes),
110         name="capacidad",
111     )
112
113     # (5) Demanda exacta de cada cliente
114     modelo.addConstrs(
115         (gp.quicksum(x[i, j] for i in origenes) == d[j] for j in destinos),
116         name="demanda",
117     )
118
119     modelo.optimize()
120
121     # ----- Reporte -----
122     if modelo.status == GRB.OPTIMAL:
123         print("Estado: OPTIMO")
124         print(f"Valor optimo: {modelo.objVal:.4f}")
125
126         print("Bodegas arrendadas:")
127         for i in origenes:

```



```

127         if y[i].X > 0.5:
128             print(f"Se decidio arrendar la bodega {i}")
129
130     print("Envios realizados:")
131     for i in origenes:
132         for j in destinos:
133             cantidad: float = x[i, j].X
134             if cantidad > 1e-6:
135                 print(
136                     f"Se enviaron {
137                         cantidad:g} unidades al cliente {j} desde la bodega {i}")
138
139     elif modelo.status in (GRB.INFEASIBLE, GRB.INF_OR_UNBD):
140         print("Estado: INFACTIBLE")
141     else:
142         print(f"Estado: OTRO (codigo Gurobi {modelo.status})")
143
144 if __name__ == "__main__":
145     main()

```

Listing 1: Contenido de main.py

Ejecución sobre la instancia de ejemplo

Sobre los archivos provistos, la ejecución de main.py produce la siguiente salida:

```

1 Estado: OPTIMO
2 Valor optimo: 204.0000
3 Bodegas arrendadas:
4   Se decidio arrendar la bodega 1
5   Se decidio arrendar la bodega 2
6   Se decidio arrendar la bodega 3
7 Envios realizados:
8   Se enviaron 8 unidades al cliente 1 desde la bodega 1
9   Se enviaron 12 unidades al cliente 2 desde la bodega 2
10  Se enviaron 3 unidades al cliente 3 desde la bodega 2
11  Se enviaron 7 unidades al cliente 3 desde la bodega 3
12  Se enviaron 9 unidades al cliente 4 desde la bodega 3

```

El valor óptimo es $z^* = 204$, correspondiente a arrendar las tres bodegas y realizar los envíos señalados.